

Class04B ANN Demo

ANN_Python_Details.ipynb

(Part 2 and 3)

```
# NEW DEMO
```

```
# Part 2: ANN with data points using one hidden Layer
```

```
#-----
```

```
# 2 inputs, one output (binary classification)
```

```
# ANN demo: 20 training instances are defined
```

```
# These points are not linearly separable
```

```
#
```

```
tmpp= np.array([
```

```
[1.0000 , 1.0000 , 1.0000],
```

```
[1.1000 , 0.6000 , 1.0000],
```

```
[1.5000 , 0.2000 , 1.0000],
```

```
[-0.5000 , 0.7500 , 1.0000],
```

```
[-0.5000 , 1.5000 , 1.0000],
```

```
[-0.1000 , 0.9000 , 1.0000],
```

```
[ 0.6000 , 1.2 , 1.0000],
```

```
[1.5000 , 1.5 , 1.0000],
```

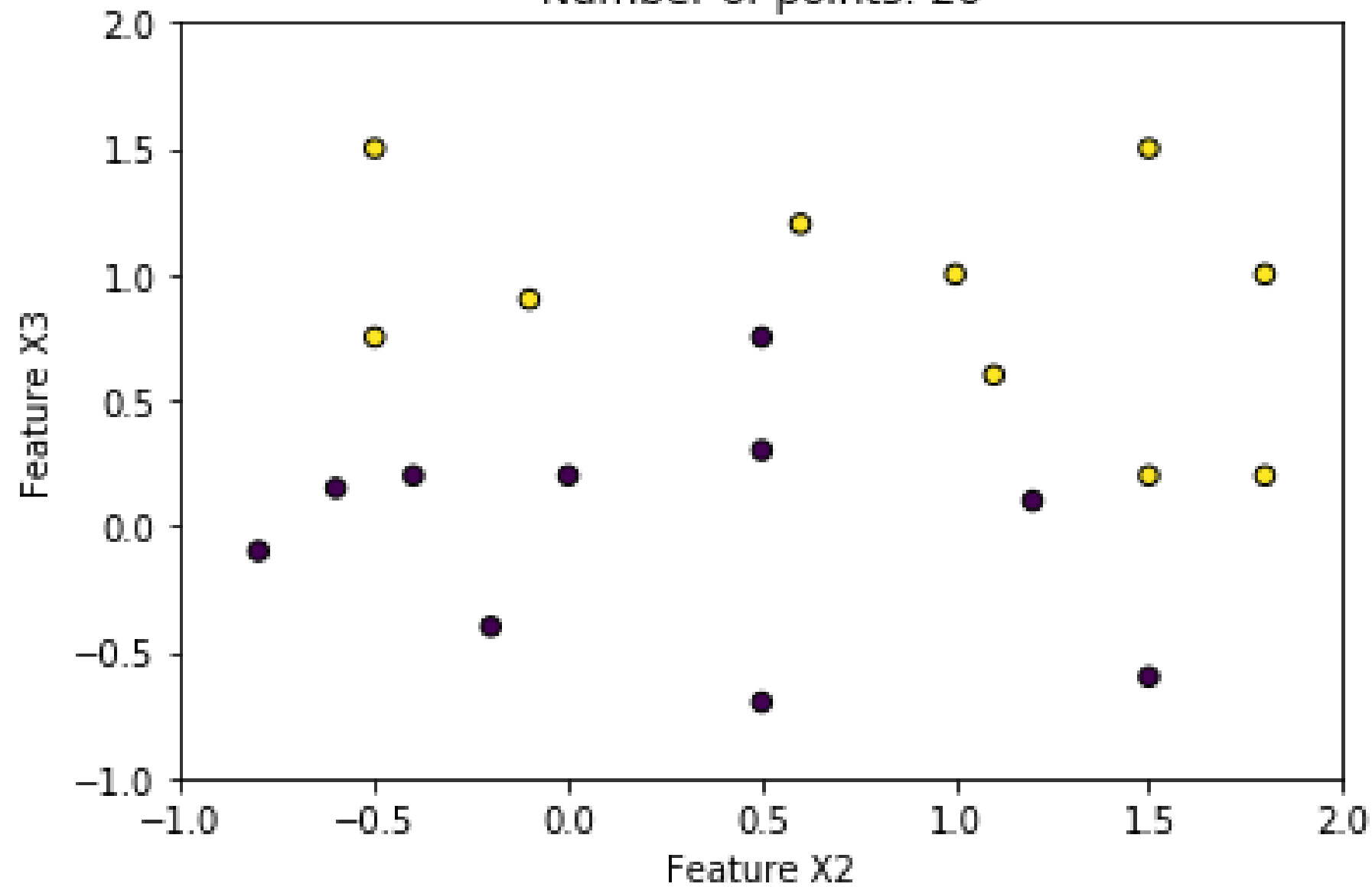
```
[1.8000 , 1.0000 , 1.0000],
```

```
[1.8000 , 0.2000 , 1.0000],
```

```
[-0.8000 , -0.1000 , 0.0000],
```

```
[ 0.4000 , 0.2000 , 0.0000]
```

Display of data points
Number of points: 20



```
train_x=tmp[0:2]
train_y=tmp[2]
train_y = train_y.reshape((20,1))
print(train_x.shape)
print(train_y.shape)
```

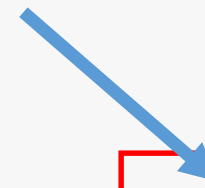
```
(20, 2)
(20, 1)
```

```
# 15 hidden units
# parameters
num_hid = 15
alpha = 0.3
beta = 0.3
err = 0.001
max_stop = 3000 # max_num_iterations
```

```
# To carry out the training
```

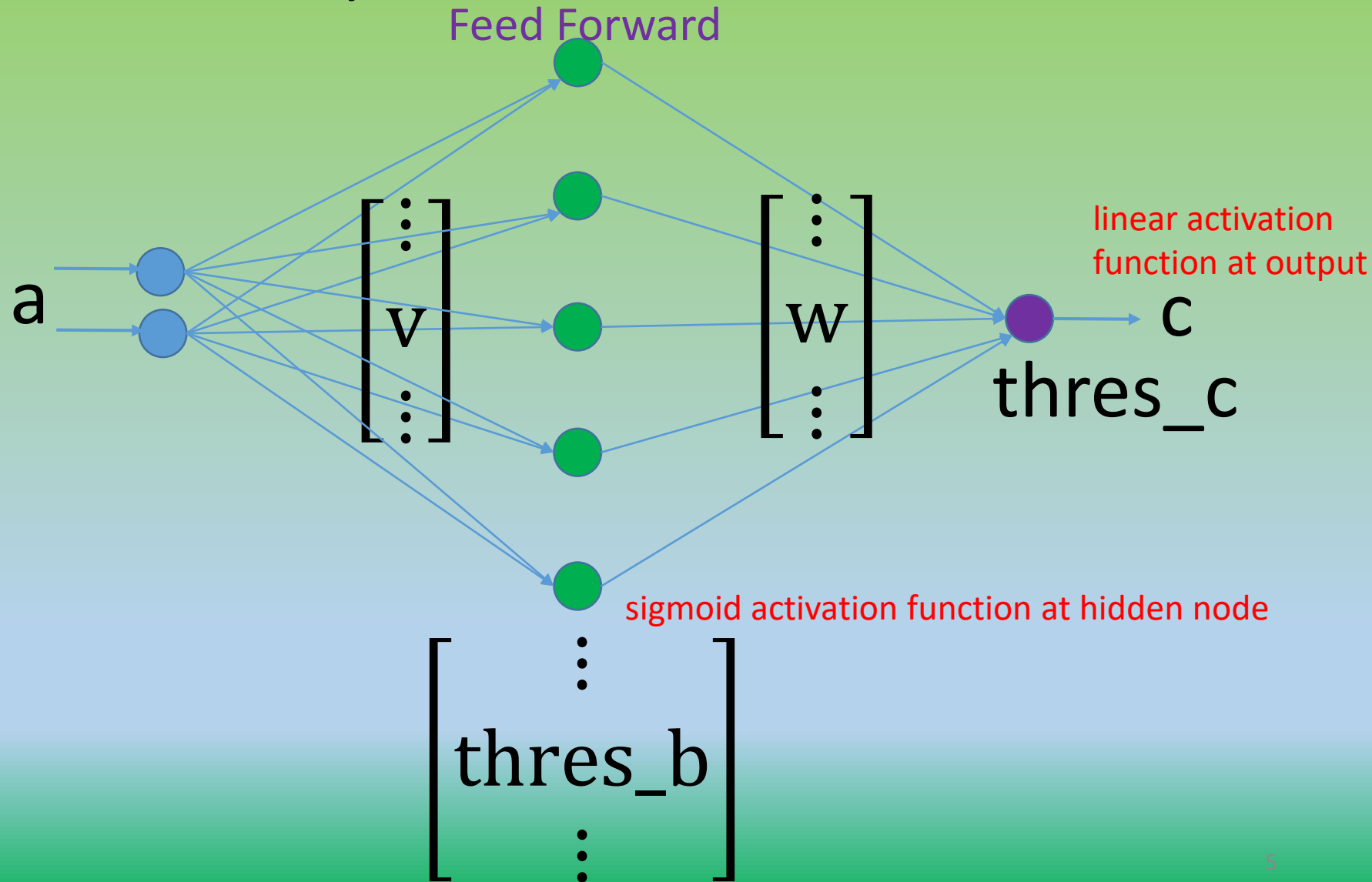
```
save_v,save_w,save_thres_b,save_thres_c,convergence, ave_err, max_err, result, v, w, thres_b, thres_c = bp_logsig_lin(train_x, t
print(convergence.shape)
```

The python function shown in Class04A



bp_logsig_lin(train_x, t

Regression Problem of **two-input**, one output,
one hidden layer.



Demo on Backpropagation Algorithm by Dr. G. Pang
number of training samples = 20
number of input = 2
number of output = 1
Stopping criterion has been reached!
(3000,)

```
print('-----')  
# Final recall  
num_examples=20  
num_out=1
```

```
result2 = np.zeros((num_examples, num_out))  
for n in range(num_examples):
```

```
    a = train_x[n, :]  
    b = sigmoid(np.dot(a, v) + thres_b)  
    result2[n, :] = np.dot(b, w) + thres_c
```

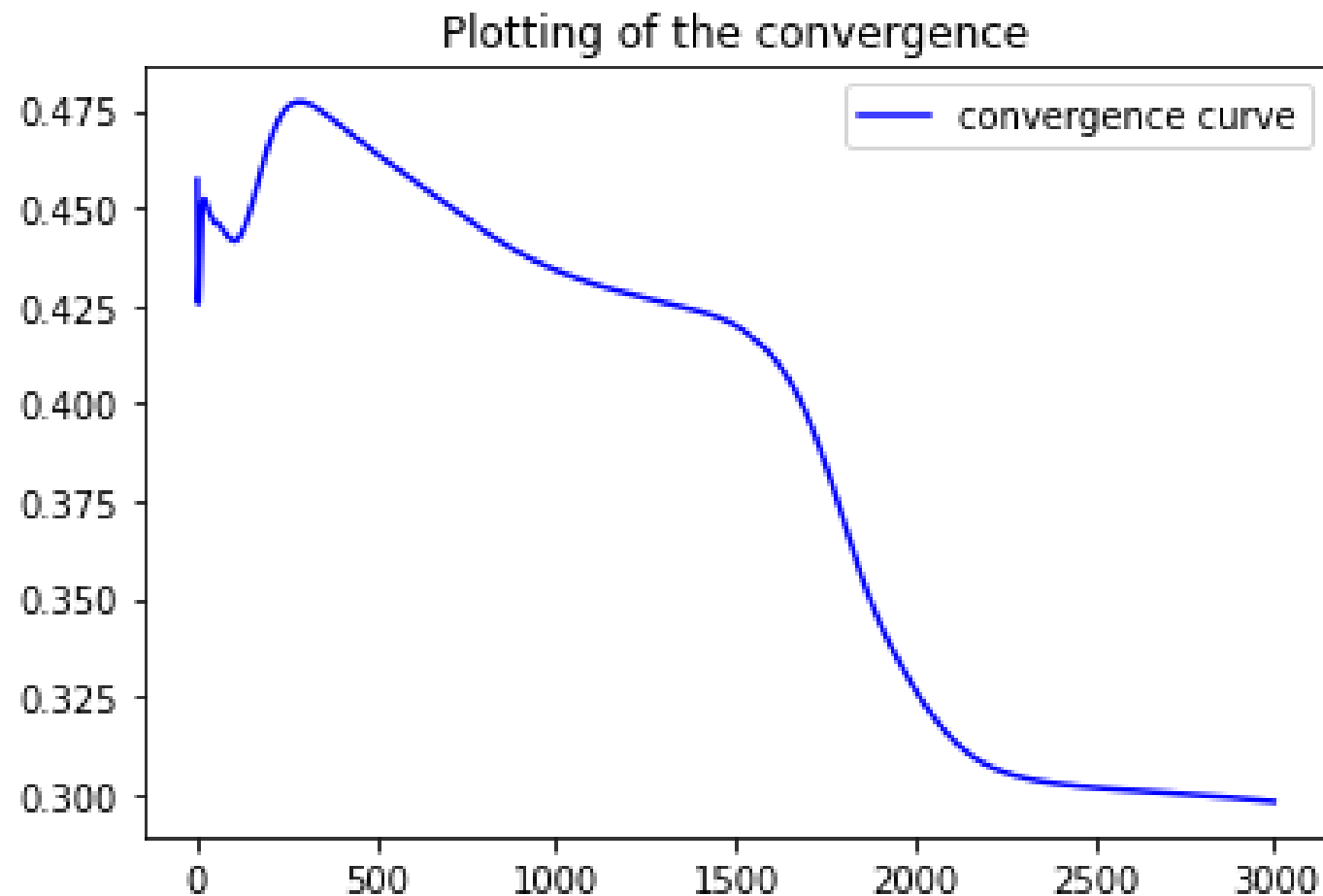
```
-----  
result2 = [[ 0.59242245]  
 [ 0.56460296]  
 [ 0.53304635]  
 [ 0.62656708]
```

Calculation at hidden layer

Feedforward

Calculation at output layer

```
# To show the convergence curve
plt.plot(np.arange(0, len(convergence)), convergence, label = 'convergence curve',
         color = 'blue')
plt.title('Plotting of the convergence')
plt.legend()
plt.show()
```



```
xx, yy = np.meshgrid(np.linspace(-1, 2, 300),  
                     np.linspace(-1, 2, 300))  
grid = np.c_[xx.ravel(), yy.ravel()]  
print(grid.shape)
```

(90000, 2)

```
# Get output for the grid points to plot decision boundaries
```

```
percept = copy.deepcopy(xx)
```

```
for ixx in range(300):
```

```
    for iyy in range(300):
```

```
        a = np.array([xx[ixx,iyy], yy[ixx,iyy]] )
```

```
        b = sigmoid(np.dot(a, v) + thres_b)
```

```
        tmp_c = np.dot(b, w) + thres_c
```

```
        percept[ixx,iyy] = np.array([tmp_c.item()]) #to convert
```

← Feedforward

```
# Get output for the grid points to plot decision boundaries
percept_binary = copy.deepcopy(xx)
```

```
for ixx in range(300):
    for iyy in range(300):
        if percept[ixx,iyy] > 0.5:
            percept_binary[ixx,iyy] = np.array(1)
        else:
            percept_binary[ixx,iyy] = np.array(0)
```

← threshold

```
print(percept_binary.shape)
```

```
(300, 300)
```

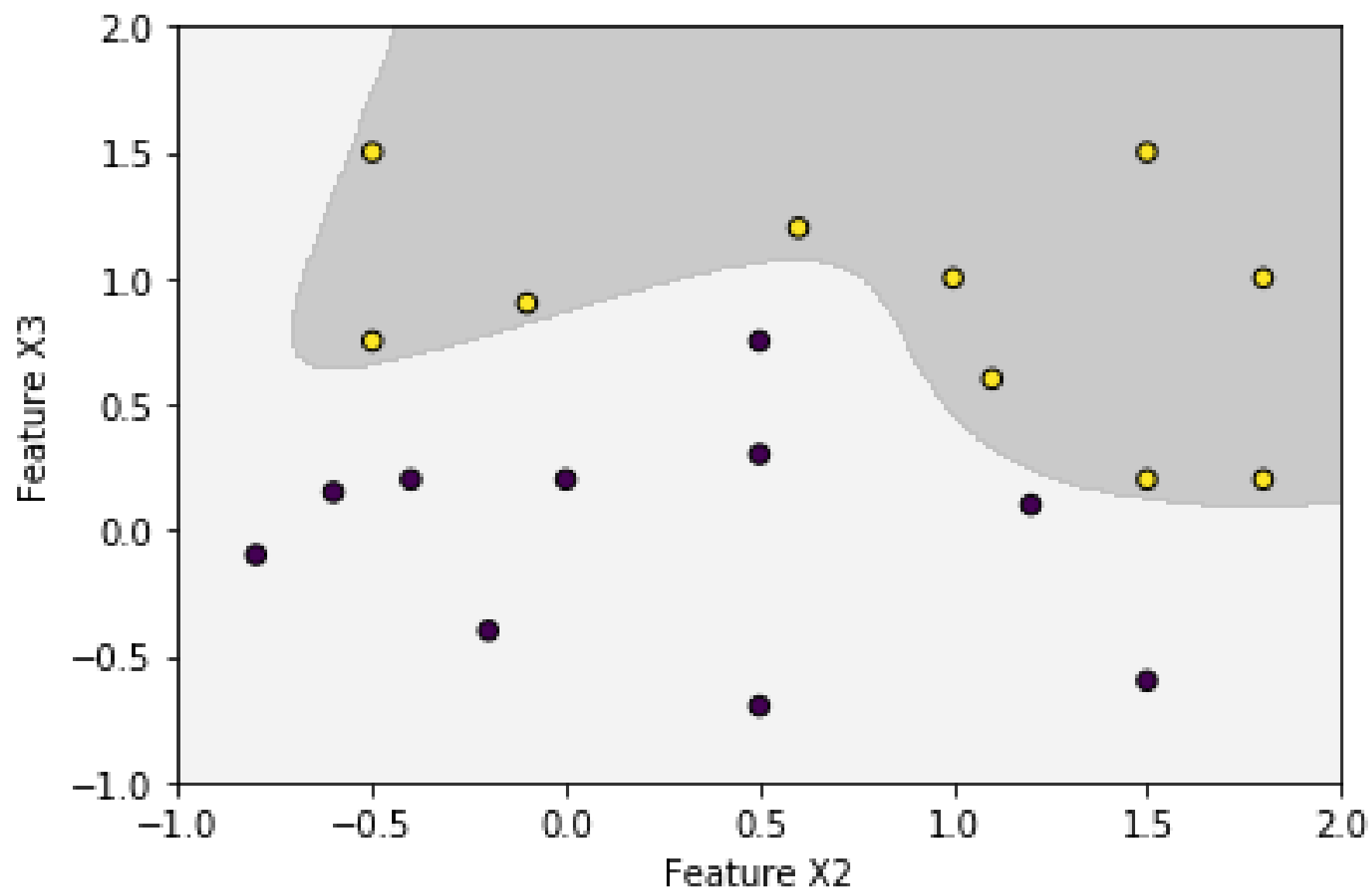
```
# Visualize the decision boundary
threshold_1 = 0.5
```

```
plt.contourf(xx, yy, percept_binary, levels=[0, threshold_1, 1], alpha=0.3, cmap=plt.cm.Greys)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)
```

```
plt.title(f'Decision Boundary with threshold {threshold_1}\n')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')
```

```
plt.show()
```

Decision Boundary with threshold 0.5



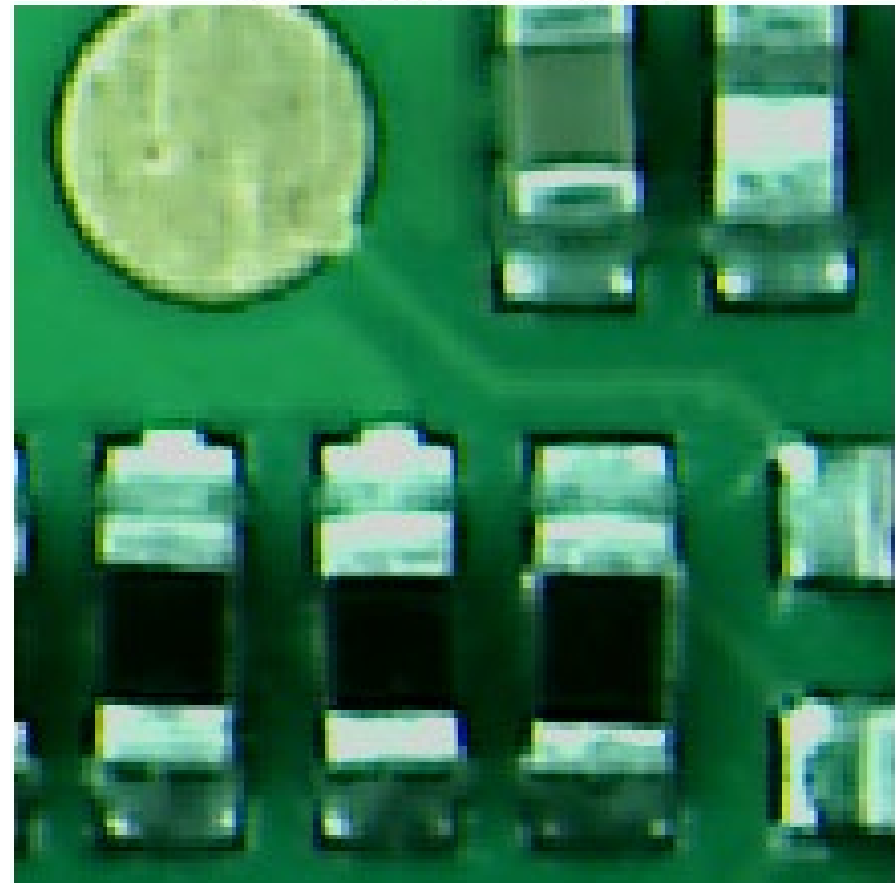
```
# Part 3: Using ANN for color image segmentation
# 3-input, 2-output classification
# segment the image into 4 colors
```

```
from PIL import Image

# Open the image file
image_name = "3503A.bmp"
img = Image.open(image_name)
# Get image dimensions
width, height = img.size
# Ensure the image is in RGB mode
img = img.convert("RGB")

# Display the image
plt.imshow(img)
plt.title("RGB Image Display")
plt.axis('off') # Hide axes
plt.show()
```

RGB Image Display



```
img_new = Image.new('RGB',(width, height),color='black')
pixels_new = img_new.load()
```

```
# Define training instances (for the 4 colors)
```

```
tmpp = np.array([
    [5, 10, 5, 0, 0],          # black
    [0, 96, 50, 255, 0],       # green
    [210, 210, 210, 0, 255],   # white
    [0, 20, 0, 0, 0],          # black
    [194, 221, 209, 0, 255],   # white
    [205, 215, 210, 0, 255],   # white
    [230, 225, 235, 0, 255],   # white
    [14, 106, 57, 255, 0],      # green
    [62, 117, 78, 255, 0],      # green
    [26, 156, 81, 255, 0],      # green
    [0, 8, 0, 0, 0],            # black
    [193, 220, 112, 255, 255],  # yellow
    [147, 192, 119, 255, 255],  # yellow
    [0, 16, 7, 0, 0],           #black
    [177, 221, 156, 255, 255],  #yellow
    [159, 204, 126, 255, 255]   #yellow
]) / 255.0
```

Define the
4 classes
of color

```
# Extract the inputs and outputs for training
train_x=tmp[0:3]
train_y=tmp[3:5]
```

```
print('train_x.shape = ', train_x.shape)
print('train_y.shape = ', train_y.shape)
```

```
# Start training with 15 hidden units
# parameters
num_hid = 15
alpha = 0.3
beta = 0.3
err = 0.001
max_stop = 7000 # max_num_iterations
```

```
# To carry out the training
```

```
save_v,save_w,save_thres_b,save_thres_c,convergence, ave_err, max_err, result, v, w, thres_b, thres_c = bp_logsig_lin(train_x, t
```

```
print('-----Finished training -----')
```

```
print(convergence.shape)
```

```
# To show the convergence curve
```

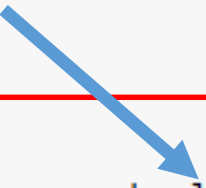
```
plt.plot(np.arange(0,len(convergence)),convergence,label = 'convergence curve',
         color = 'blue')
```

```
plt.title('Plotting of the convergence')
```

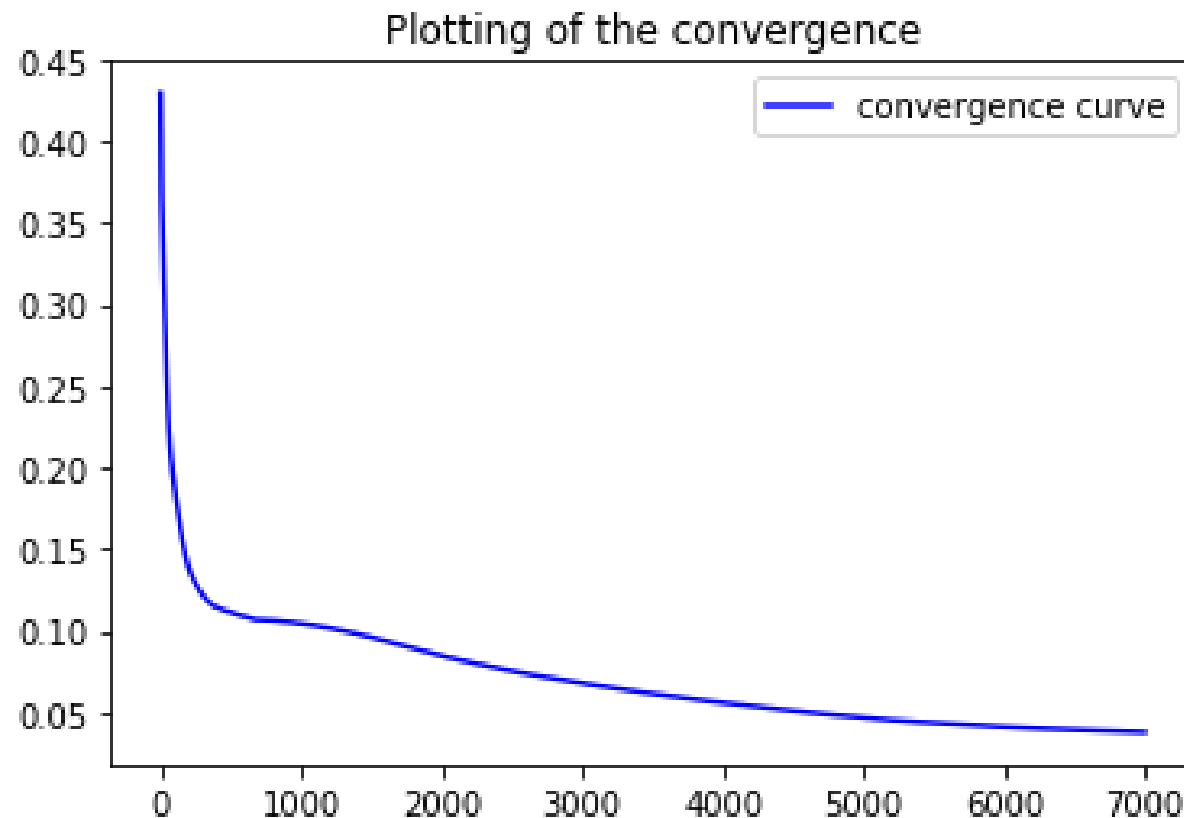
```
plt.legend()
```

```
plt.show()
```

The python function shown in Class04A



```
train_x.shape = (16, 3)
train_y.shape = (16, 2)
Demo on Backpropagation Algorithm by Dr. G. Pang
number of training samples = 16
number of input = 3
number of output = 2
Stopping criterion has been reached!
-----Finished training -----
(7000,)
```



```

# To create a new image
black_v = [0, 0, 0]
white_v = [230, 230, 230]
green_v = [0, 100, 50]
yellow_v = [150, 190, 120]

# Iterate over each pixel in the image
width, height = img.size
threshold_d = 0.5

for x in range(height):
    for y in range(width):
        # Extract R, G, B values for the current pixel
        R, G, B = img.getpixel((x, y))

        a = np.array([R, G, B])/255 # scale down the input R,G,B, pixels
        b = sigmoid(np.dot(a, v) + thres_b)
        result_c = np.dot(b, w) + thres_c

        out1 = 1 if result_c[0,0] > threshold_d else 0
        out2 = 1 if result_c[0,1] > threshold_d else 0

        # Assign colors based on the outputs
        if out1 == 0 and out2 == 0:
            pixels_new[x,y] = (0,0,0) # Black
        elif out1 == 0 and out2 == 1:
            pixels_new[x,y] = (255,255,255) # White
        elif out1 == 1 and out2 == 0:
            pixels_new[x,y] = (0,100,50) # Green
        elif out1 == 1 and out2 == 1:
            pixels_new[x,y] = (150,190,120) # Yellow

```

Feedforward

4 classes
(shown by
4 colors)

```
# Display the image  
plt.imshow(img_new)  
plt.title("RGB Image Display")  
plt.axis('off') # Hide axes for a cleaner image display  
plt.show()
```

RGB Image Display



```
# To show the curves of four parameters
plt.plot(np.arange(0,len(convergence)),save_v,label = 'v value',color = 'blue')

plt.plot(np.arange(0,len(convergence)),save_w,label = 'w value',color = 'green')

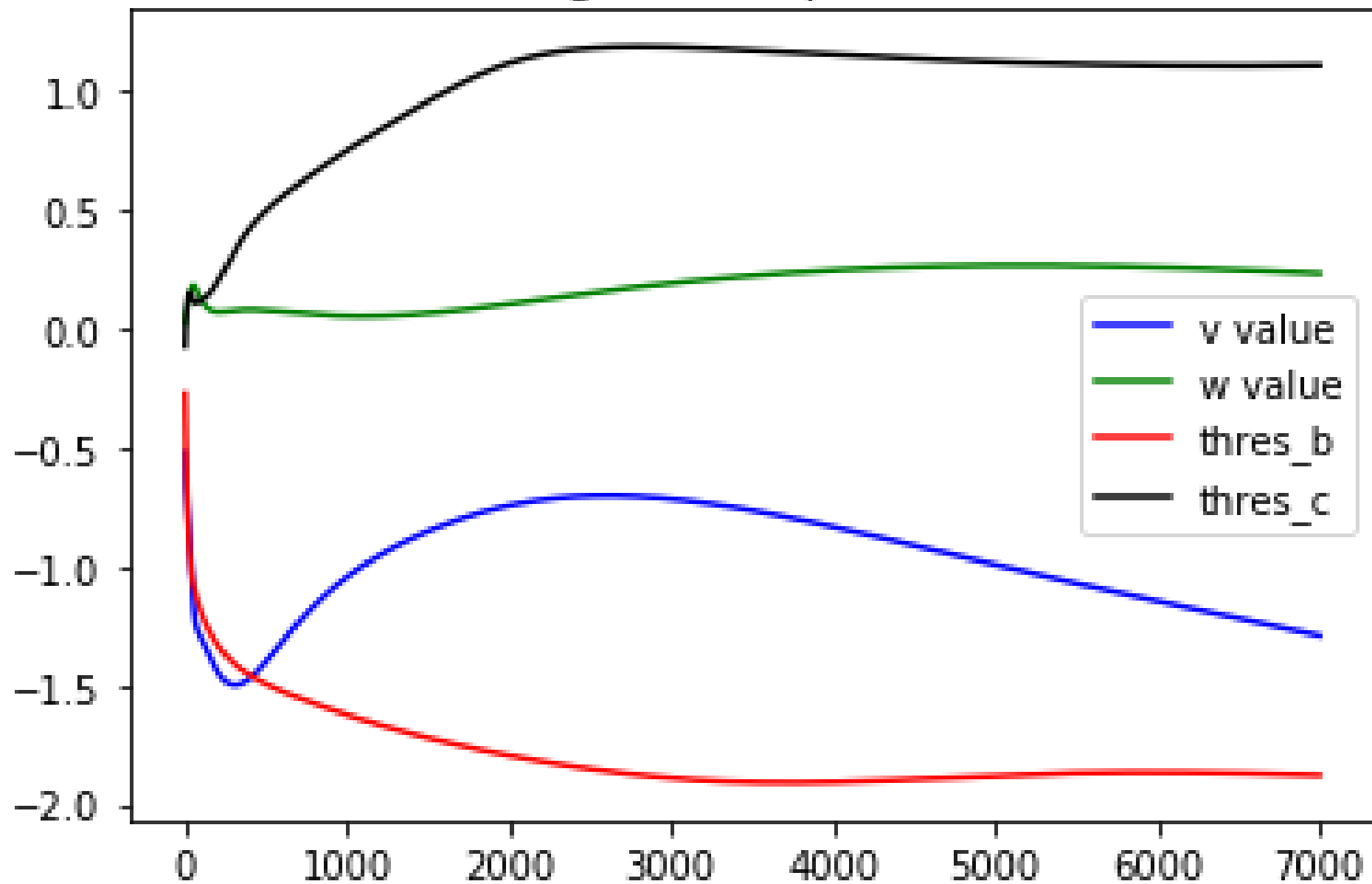
plt.plot(np.arange(0,len(convergence)),save_thres_b,label = 'thres_b',color = 'red')

plt.plot(np.arange(0,len(convergence)),save_thres_c,label = 'thres_c',color = 'black')

plt.title('Plotting of some parameters ')
plt.legend()
plt.show()

print('v =', v)
print('w =', w)
print('thres_b =', thres_b)
print('thres_c =', thres_c)
```

Plotting of some parameters



```
from PIL import Image

# Open the image file
image_name = "sunset.jpg"
img = Image.open(image_name)
# Get image dimensions
width, height = img.size
# Ensure the image is in RGB mode
img = img.convert("RGB")
print(width, height)
# Display the image
plt.imshow(img)
plt.title("RGB Image Display")
plt.axis('off') # Hide axes
plt.show()
```

1284 1081

RGB Image Display



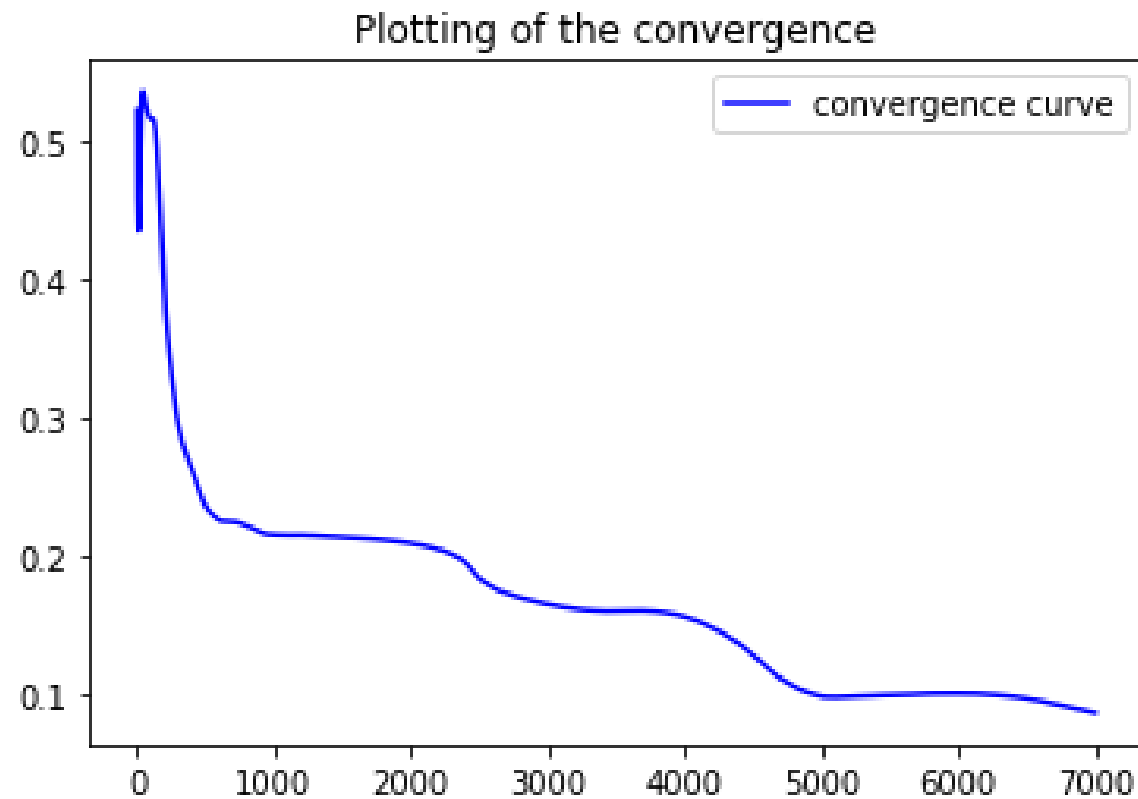
```

img_new2 = Image.new('RGB',(width, height),color='black')
pixels_new2 = img_new2.load()

# Define training instances (for the 8 colors)
tmpp = np.array([
    [199,91,39, 0, 0, 0],      # orange1
    [191,95,45, 0, 0, 0],      #
    [188, 87, 43, 0, 0, 0],      #
    [239,129, 42,0,0, 255],      # light_orange
    [241,133, 34,0,0, 255],      #
    [238,125, 57,0,0, 255],      #
    [143,62,45,0,255,0],        # dark_orange
    [139,67,55,0,255,0],        #
    [148,62,49,0,255,0],        #
    [240,48,0,0,255,255],        # orange2
    [242,52,2,0,255,255],        #
    [241,49,2,0,255,255],        #
    [240,235,4,255,0,0],        # yellow
    [241,206,40,255,0,0],        #
    [241,210,32,255,0,0],        #
    [243,28,0,255,0,255],        # orange3
    [240,9,1,255,0,255],        #
    [236,12,0,255,0,255],        #
    [72,49,43,255,255,0],        # grey1
    [75,54,51,255,255,0],        #
    [60,50,48,255,255,0],        #
    [51,33,33,255,255,255],      # dark_grey
    [70,64,64,255,255,255],      #
    [51,51,49,255, 255,255]      #
]) / 255.0

```

```
train_x.shape = (24, 3)
train_y.shape = (24, 3)
Demo on Backpropagation Algorithm by Dr. G. Pang
number of training samples = 24
number of input = 3
number of output = 3
Stopping criterion has been reached!
-----Finished training -----
(7000,)
```



```
# To create a new image
orange1_v = [199,91,39]
light_orange_v = [239,129,42]
dark_orange_v = [143,62,45]
orange2_v = [240,48,0]
yellow_v = [240,235,4]
orange3_v = [243,28,0]
grey1_v = [72,49,43]
dark_grey_v = [51,33,33]

# Iterate over each pixel in the image
width, height = img_new2.size
threshold_d = 0.6

print('width = ', width)
print('height = ', height)

for x in range(width):
    for y in range(height):
        # Extract R, G, B values for the current pixel of given image
        R, G, B = img.getpixel((x, y))

        a = np.array([R, G, B])/255 #Needs to scale down the input R,G,B to [0,1]
        b = sigmoid(np.dot(a, v) + thres_b)
        result_c = np.dot(b, w) + thres_c

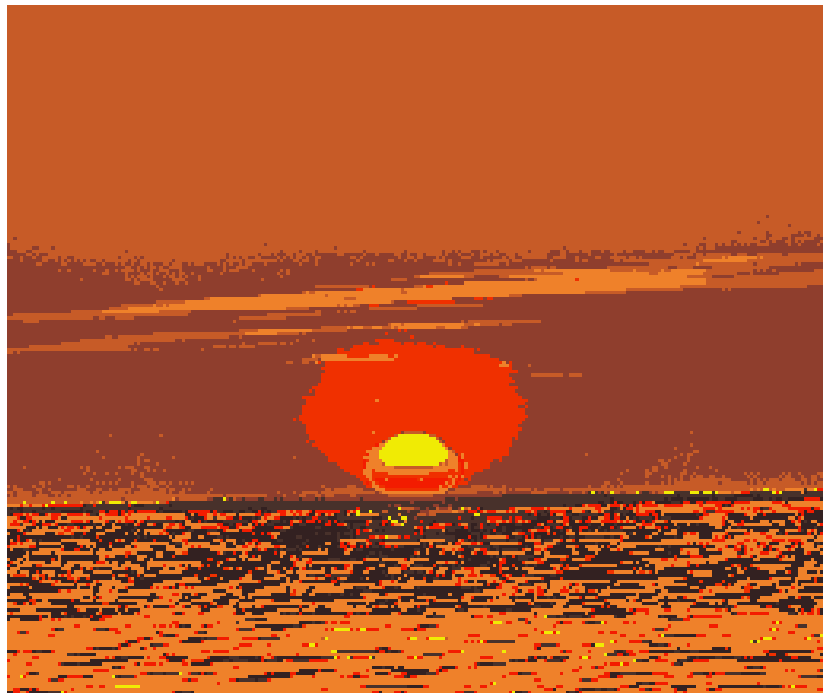
        out1 = 1 if result_c[0,0] > threshold_d else 0
        out2 = 1 if result_c[0,1] > threshold_d else 0
        out3 = 1 if result_c[0,2] > threshold_d else 0
```

```
# Assign colors based on the outputs
if out1 == 0 and out2 == 0 and out3 == 0:
    pixels_new2[x,y] = (199,91,39)      # orange1
elif out1 == 0 and out2 == 0 and out3 == 1:
    pixels_new2[x,y] = (239,129,42)     # Light_orange
elif out1 == 0 and out2 == 1 and out3 == 0:
    pixels_new2[x,y] = (143,62,45)      # dark_orange
elif out1 == 0 and out2 == 1 and out3 == 1:
    pixels_new2[x,y] = (240,48,0)       # orange2
elif out1 == 1 and out2 == 0 and out3 == 0:
    pixels_new2[x,y] = (240,235,4)      # yellow
elif out1 == 1 and out2 == 0 and out3 == 1:
    pixels_new2[x,y] = (243,28,0)       # orange3
elif out1 == 1 and out2 == 1 and out3 == 0:
    pixels_new2[x,y] = (72,49,43)       # grey1
elif out1 == 1 and out2 == 1 and out3 == 1:
    pixels_new2[x,y] = (51,33,33)       # dark_grey
```

```
# Display the image
plt.imshow(img_new2)
plt.title("RGB Image Display")
plt.axis('off') # Hide axes for a cleaner image display
plt.show()
```

```
width = 1284
height = 1081
```

RGB Image Display



RGB Image Display

